/ MMB8052 - Practical 8

MMB8052 Practical 08 - Case Study III: RNA Sequencing Analysis

Introduction

The analysis of gene expression data, generated by an RNA Sequencing (RNA-Seq) experiment, is one of the most common bioinformatics applications. Transcriptomics technologies have seen a lot of development over the last 25 years, and RNA-Seq is just the latest in a long line of techniques for assaying gene expression on a whole genome scale. The techniques for robust data analysis have developed alongside the experimental technologies.

A note on this practical, and the assessment

For this practical, I have provided all of the code required to complete the exercises. This is the practical that forms the basis of the second assessment, and so it is important that you can complete the exercises as they are documented here. It is also possible to download the code for completing these exercises from the module Github repository.

RNA Sequencing

RNA Sequencing experiments take advantage of a simple assumption – that the number of reads that map to a particular molecule in a next generation experiment is in direct proportion to both its length, and the amount of that molecule that is present in the sample. So if two RNAs of equal length are present at a ratio of 2:1 in two samples, then we will retrieve twice as many reads for one as we will for the other when we sequence the samples. So, given two samples we can normalise for transcript lengths and total number of reads, and work out which species are differentially expressed between the two samples.

RNA-Seq is often seen as a replacement for the very popular microarray approach to studying gene expression, and while the two approaches have different strengths and weaknesses, it is generally true that the data from RNA-Seq correlates better with qPCR data (the “gold standard” for measuring gene expression data) and the sheer depth of the data generated makes RNA-Seq the better choice in many circumstances (assuming the budget allows).

This also means that many of the mistakes, with regards to experimental design, that were made in the early days of microarrays have been repeated in earnest with RNA-Seq experiments. One of the most common false assumptions is that with RNA-Seq you don't need replicates. This is simply untrue, we still need to model the variance of the population in order to return meaningful statistics and consequently it's advisable to do at least 4 biological replicates per condition minimum for cell lines, more for mice/rats and many more for humans. Unlike with microarray analysis however technical replicates are seldom required, as despite the stochastic nature of sequencing runs they tend to be highly reproducible.

RNA-Seq Analysis

After quality assessment (not covered here, but important), RNA-Seq analysis usually comprises 3 (or 4) steps:

1. Align sequencing reads to a reference genome (this step is increasingly optional)
2. Quantify RNA expression against reference gene annotation
3. Analyse RNA counts to define differential expression
4. Functional analysis of differentially expressed genes

Here, we will be focussing on step 3, which is the bulk of the analysis. The final case study in the module will cover functional analysis – which can be applied downstream of many analysis tasks (not just RNA-Seq).

The data we will be working with, and the process for downloading and processing the data to make it ready for step 3 are available from the module's Github page: https://github.com/sjcockell/mmb8052/tree/main/practicals/practical_08

RNA-Seq Read Alignment

We covered alignment of high-throughput sequencing reads to a reference genome in practical 7, and much of what we discussed there holds true for RNA-Seq alignment (the need for efficiency as experiments contain millions of reads, the nature of the alignment problem etc) with one key difference. The constructs in the sequencing library are derived from RNA, and in the case of eukaryotic mRNA that RNA is spliced post-transcription. Splicing means that a single read sequence may be derived from more than one genomic location, often separated by thousands of bases because of an intervening intron which has been spliced out. A read aligner designed for RNA-Seq data therefore has to be able to cope with split reads which arise because of splicing.

Examples of splice-aware read aligners include:

- STAR - https://doi.org/10.1093/bioinformatics/bts635
- HISAT2 - https://doi.org/10.1038/s41587-019-0201-4
- Tophat2 - https://doi.org/10.1186/gb-2013-14-4-r36

Quantification of RNA-Seq Data

Modern RNA-Seq analysis procedures tend to avoid the alignment step, which is computationally costly, slow, and results in very large output files. These output files are largely redundant, given that they replicate the contents of the raw FASTQ files.

Rather than alignment, tools which quantify transcript expression via read abundances tend to be favoured in modern analysis. We will be working with data produced by Salmon, a tool which performs *quasi-alignment* to assign sequencing reads to their transcript of origin. The process by which the quantification estimates we will be working with have been produced will be described below.

Quasi-alignment

Instead of the more ‘traditional’ approach of a full read alignment to the reference genome, Salmon uses an efficient, lightweight alignment methodology to assign the reads in an RNA-Seq experiment to their parent transcripts. It then uses these assignments directly to estimate the abundance of each transcript from the reference annotation in the sample.

It is possible for reads to be assigned to more than one transcript – since many transcripts from one gene can share exons. These ambiguous reads are assigned fractionally based on abundance estimates for the possible transcripts made using uniquely assigned reads. For this reason, despite read assignment looking like a discrete, counting problem, we end up with fractional transcript abundances.

Annotation

There are many sources of gene annotation available. Even for well-annotated organisms, these resources vary in the approaches they take to annotation, and therefore how complete (or otherwise) they appear.

The annotation used in an RNA-Seq experiment is one of the biggest factors effecting the outcome of the data analysis. The choices available largely depend on the organism that you're working with. For human and mouse experiments, GENCODE is recommended. The GENCODE gene sets are a combination of the HAVANA manual annotation, and the Ensembl automatic annotation pipeline. The goal is to achieve an accurate and complete annotation of the human (and mouse) genome.

For other organisms, I would recommend working with the Ensembl annotation, where possible. The quality of Ensembl gene sets is consistent, and the data is easily available. The Ensembl FTP server is a convenient resource for this data for many organisms: http://www.ensembl.org/info/data/ftp/index.html

For this example experiment, the data was derived from mouse cells, so we can download the latest GENCODE mouse gene set, using wget:

```
$ wget -O extdata/gencode.vM31.transcripts.fa.gz htt
```

This FASTA sequence file is then indexed for use with Salmon. This is similar to the genome indexing carried out for alignment - a one-time, reasonably intensive procedure which can be stored and used multiple times.

```
$ salmon index -t extdata/gencode.vM31.transcripts.f
```

Quantification

Since the raw RNA-Seq data we are working with is large (12GB), and the process of quantification still relatively time-consuming (in the context of a 3-hour practical), the quantification has already been completed. The general process for this is to use `salmon quant` for each sample:

```
for sample in sample_list
do
    salmon quant -l A -r data/${sample}.fastq.gz -i
done
```

This produces a file (`quant.sf`) for each sample, which contains the transcript-level quantification data for that sample. Below is an abbreviated version of this data for five transcripts:

```
Name                    Length  EffectiveLength TPM
ENSMUST00000162795.8    2712    2462.000        1.61
ENSMUST00000161327.8    5707    5457.000        4.56
ENSMUST00000159802.2    4136    3886.000        0.20
ENSMUST00000162257.8    2204    1954.000        4.08
ENSMUST00000159206.2    519     269.000         2.58
```

The Dataset - GSE116583

For the exercises in this practical, we will be using a publicly available RNA-Seq dataset from the Gene Expression Omnibus (GEO). This dataset is from murine alveolar macrophages early after reperfusion (restoration of blood flow once the organ is in place in the receipient) in a model of lung transplantation. The data is described in the GEO entry: GSE116583. There's lots of information about the experiment which generated the data in the so-called SOFT format file which you can download from the entry: https://ftp.ncbi.nlm.nih.gov/geo/series/GSE116nnn/GSE116583/soft/GSE1 This can be quite hard to understand, so I've extracted some key information in the table below.

Metadata Header	Experiment Information
Timepoints	Naive (control), 2 hours post-reperfusion, 24 hours post-reperfusion (n = 4)
Sample Extract Protocol	PicoPure (Thermofisher) RNA isolation kit. RNA libraries were prepared for sequencing using standard Illumina protocols.
Mouse Genotype	Cx3cr1$^{gfp/+}$B6
Mouse Sex	Male
Mouse Age	14 weeks
Tissue	Lung
Cell Type	alveolar macrophage

Metadata Header	Experiment Information
RNA-Seq type	polyA RNA
Sequencer	NextSeq 500
Library layout	Single-end
Read Length	75 bp

As is often the case with publicly available data, the amount of information on how the experiment was carried out is quite sparse. We can learn more about the role of alveolar macrophages (and macrophages more widely) in organ transplantation here: https://doi.org/10.1111/ajt.15751. The lab which produced the data in GSE116583 has documented their mouse lung transplant protocol in this work: https://doi.org/10.1126/scitranslmed.aal4508 (protocol information is in the Supplementary Materials).

I have downloaded the FASTQ data for the 12 samples in this experiment, and used Salmon as documented above to quantify the transcript-level expression in each sample.

The `quant.sf` files which result from this process can be downloaded here:
https://github.com/sjcockell/mmb8052/raw/main/practicals/practical_08/re
The full process and documentation, and a sample table can be viewed in the module's Github page, see here:
https://github.com/sjcockell/mmb8052/tree/main/practicals/practical_08.
Finally, a sample table (which we will use in the exercises below) can be downloaded from here:
https://raw.githubusercontent.com/sjcockell/mmb8052/main/practicals/pra

Exercise 8.1

Estimated time: 10 minutes

- Install and load all the packages we will need for this practical:

```
> install.packages('BiocManager')
> library(BiocManager)
> install(c('tximport', 'DESeq2', 'biomaRt', 'pheatm
> library(tximport)
> library(DESeq2)
> library(biomaRt)
> library(pheatmap)
> library(tidyverse)
```

- The first line of the above code installs Bioconductor (an R package manager for bioinformatics). The subsequent `install()` command uses the Bioconductor installer to locate the rest of the required packages.

- Bioconductor packages are not available in CRAN, so cannot be installed with `install.packages()`. The Bioconductor installer, on the other hand, is capable of installing CRAN packages (such as `pheatmap`, here).

- Some of these packages may take some time to install (particularly `DESeq2` which has a lot of dependencies). If you find yourself waiting, read the next section and come back in a few minutes to check if the installation has worked.

tximport

Reading data from a large number of files into a single data frame is not very convenient - fortunately, `tximport` is a Bioconductor package that makes this simple for the data produced by Salmon. It also summarises Salmon's transcript-level counts to a gene-level abundance estimate. This gene-level estimate is much more suitable for conventional RNA-Seq analysis.

For `tximport`, we need three pieces of information:

1. a vector of the names of the count files on our file system
2. that vector needs `names()` which match the sample names
3. a `data.frame` (or `tibble`) that maps the transcript IDs in the count file to their parent gene IDs

We can generate the first two using the sample table mentioned above, along with some of the helpful Tidyverse packages that were introduced during practical 5. The third piece of information is provided in an addtional file.

Exercise 8.2

Estimated time: 5 minutes

- Download the count data https://github.com/sjcockell/mmb8052/raw/main/practicals/practical_
- Put this .zip in your R working directory and unzip it
- Run the following R code to read the count data:

```
> sample_table = read_csv('https://raw.githubusercon
> files = pull(sample_table, Run)
```

```
> files = paste0('counts/', files, '/quant.sf')
> names(files) = pull(sample_table, Run)
> gene_map = read_csv('https://github.com/sjcockell/
> txi = tximport(files,
                 type='salmon',
                 tx2gene=gene_map,
                 ignoreTxVersion=TRUE)
```

Consider the following:

- What does the `files` vector contain at the end of this process? What if your count files where elsewhere on the file system?
- What type of variable is `gene_map` and what data is contained within it?
- What does the `ignoreTxVersion` argument for `tximport` do? Can you figure out why this is required?

DESeq2

`DESeq2` is a specialised package for the analysis of RNA-Seq data - it uses a model based on the negative binomial distribution for testing differential expression.

`DESeq2` provides normalisation and stabilisation procedures for processing the count output from Salmon. The simplest implementation of `DESeq2` is to simply create a dataset and then run the `DESeq()` method over it, but we are going to break this down into the separate steps for greater clarity of what's actually happening.

Normalisation

`DESeq2` normalises experiments imported from Salmon by a combination of *size factors* (which correct read counts based on the size of the sequencing library) and *gene length*. Size factors are calculated using the median-of-ratios method. Briefly, for each row in the counts matrix, `DESeq2` calculates the geometric mean of the row, and then divides the observed counts by that geometric mean. The size factor for a column (i.e. sample) is the median of these $(observedcounts/geometricmeans)$ across the whole sample. For data imported via `tximport`, DESeq2 also uses the per-gene average transcript length to calculate a normalisation factor for every observation in the experiment, which combines size factor normalisation with average transcript length normalisation. These normalisation factors can be retrieved using the `normalizationFactors()` method.

Dispersion Estimates

The *dispersion* is a genewise measure of the variance of the count data. It is used in `DESeq2` to model the *biological noise* in the system. The variance (which is used by the statistical test performed in the next step) is the sum of this measure of biological variance and the shot noise, which is a measure of the uncertainty of read counting as a quantitative metric. For highly expressed genes, the biological noise dominates, for lowly expressed genes it's the shot noise that dominates.

In `DESeq2` (in contrast to the older `DESeq`), the dispersion estimates are submitted to empirical Bayes shrinkage. The full details of this, in all their impenetrable, statistical glory, are available in the `DESeq2` paper (as you may notice from the title of that paper, fold change estimates are also shrunk by a related methodology).

Dispersion estimates can also be plotted (see Figure 1). This can be a useful diagnostic plot for an RNA-Seq experiment - the kind of relationship seen in this experiment between mean count and dispersion is typical, where variance decreases with increasing read count. A deviation from this usual relationship could be cause for concern for your analysis.

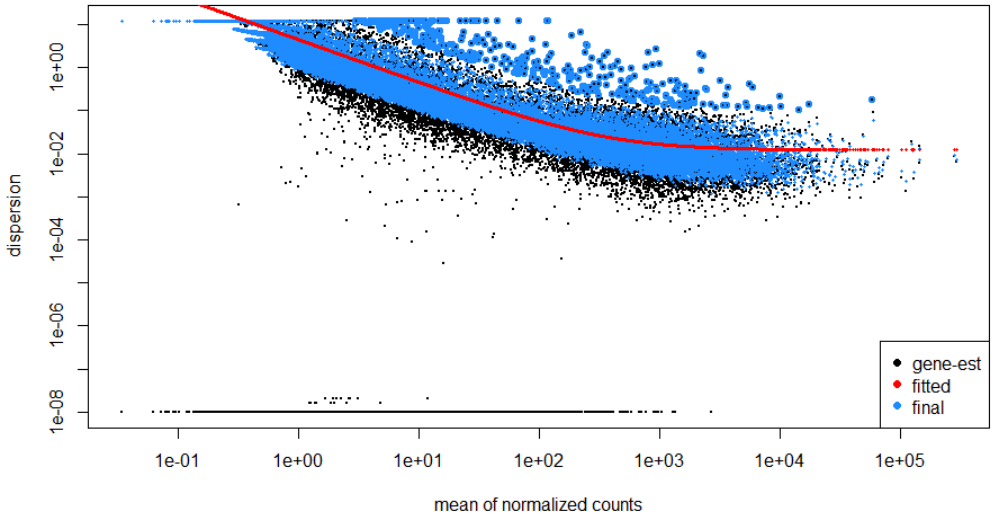


Figure 1: Plot of dispersion estimates for GSE116583.

Applying Statistics

`DESeq2` uses Wald's Negative Binomial test to determine whether genes are differentially expressed. This test is appropriate for the discrete nature of RNA-Seq data, and the distribution it typically fits. At this point, we have got as far as the convenience method `DESeq()` would take us.

Exercise 8.3

Estimated time: 10 minutes

- Run the following R code to carry out the steps described above:

```
> dds = DESeqDataSetFromTximport(txi, colData = samp
> dds = estimateSizeFactors(dds)
> dds = estimateDispersions(dds)
> dds = nbinomWaldTest(dds)
# the above 3 commands can be replaced by dds = DESe
> plotDispEsts(dds)
```

- You can access the data from the `dds` object (a `DESeqDataSet`) by using so-called accessor methods:
    - `counts()` returns the count data, the `normalized` argument can be used to provide raw or normalised counts
    - `colData()` returns the sample table (the data about the samples - i.e. the columns)
    - `normalizationFactors()` returns a matrix of normalisation factors with the same dimensions as the expression data.
- Compare `counts(dds, normalized=FALSE)/normalizationFactors(dds)` with `counts(dds, normalized=TRUE)`

Data Quality Control

Before we examine the results of these tests for differentially expressed genes, this is a good point to do some QC on our count data, to reveal any potential problems and take steps to correct them.

Many of the common methods for exploring multidimensional data, especially clustering and ordination methods such as Principal Components Analysis, do not work well with RNA-Seq data (this is due to things like skedasticity, which is a bit out of scope here). `DESeq2` provides some (relatively) simple data transformations that make data amenable to these analyses.

Regularized Logarithm Transformation

PCA with normalised read counts will be dominated by the most strongly expressed genes which exhibit the greatest absolute differences between samples. A regular log2 transformation will lead to the most weakly expressed genes dominating, due to the large amount of noise inherent in these observations. The 'regularized logarithm transformation' (`rlog()`) offered by DESeq2 corrects the first problem, and controls for the second (using empirical Bayes techniques). This transformed data can be used for PCA etc., but should not be used to test for differential expression.

Variance Stabilizing Transformation

For very large datasets, `rlog()` is prohibitively slow, and it is also less sensitive to size factors and therefore is inappropriate for cases where library size varies widely. For these cases, `DESeq2` also offers a Variance Stabilizing Transformation (`varianceStabilizingTransformation()`), which broadly achieves the same ends but is much faster.

Principal Components Analysis (PCA)

Principal Components plots can be extremely useful in determining whether any of our samples is an outlier, and in examining whether the relationships between our samples are as we would expect. In general, we would expect samples from the same experimental group to behave similarly in a PCA - so when plotting the results of the analysis, sample groups should be seen to cluster. Any exceptions to this pattern are considered *outliers*, and should be considered for removal from the analysis (for data quality reasons).

Sample Distance Analysis

Another method for identifying samples which behave unusually is to examine the statistical distance of samples from one another. There are many measures of distance, but the one used most often for data like RNA-Seq data is *Euclidian distance*. The distance of all of the samples from each other, once calculated, can be plotted in a heatmap - outlier samples should be easy to spot in this format.

Exercise 8.4

Estimated time: 10 minutes

- Run the following R code to carry out the steps described above:

```
> rld = rlog(dds)
> # vst = varianceStabilizingTransformation(dds)
> plotPCA(rld, intgroup='Group')
> sample_distance = dist(t(assay(rld)), method='eucl
> sample_distance_matrix = as.matrix(sample_distance
> heatmap_annotation = data.frame(group=colData(dds)
> pheatmap(sample_distance_matrix,
```

```
          clustering_distance_rows=sample_distance,
          clustering_distance_cols=sample_distance,
          annotation_col = heatmap_annotation)
```

- Are there any outliers in this experiment?
- Have a look at `heatmap_annotation` - what's the purpose of this object?

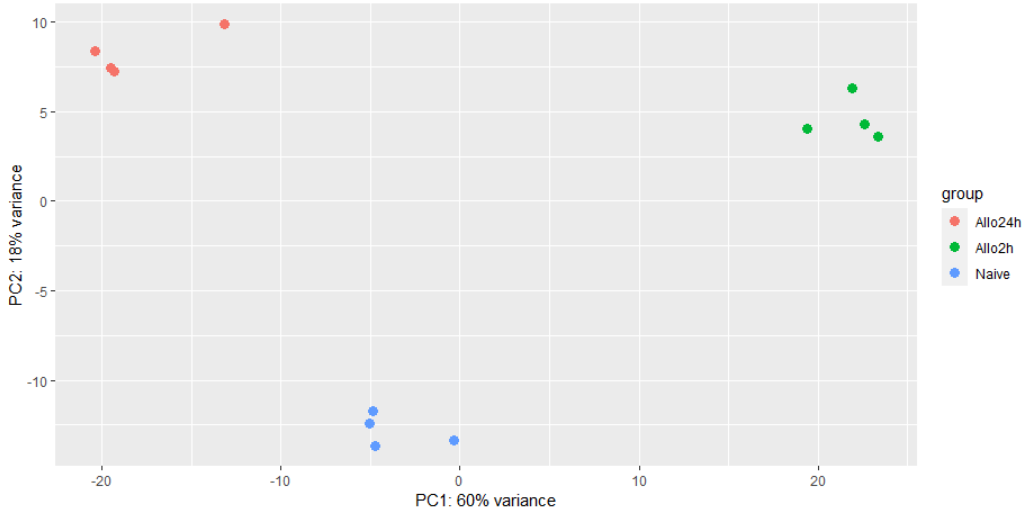


Figure 2: Plot of principal components for GSE116583.

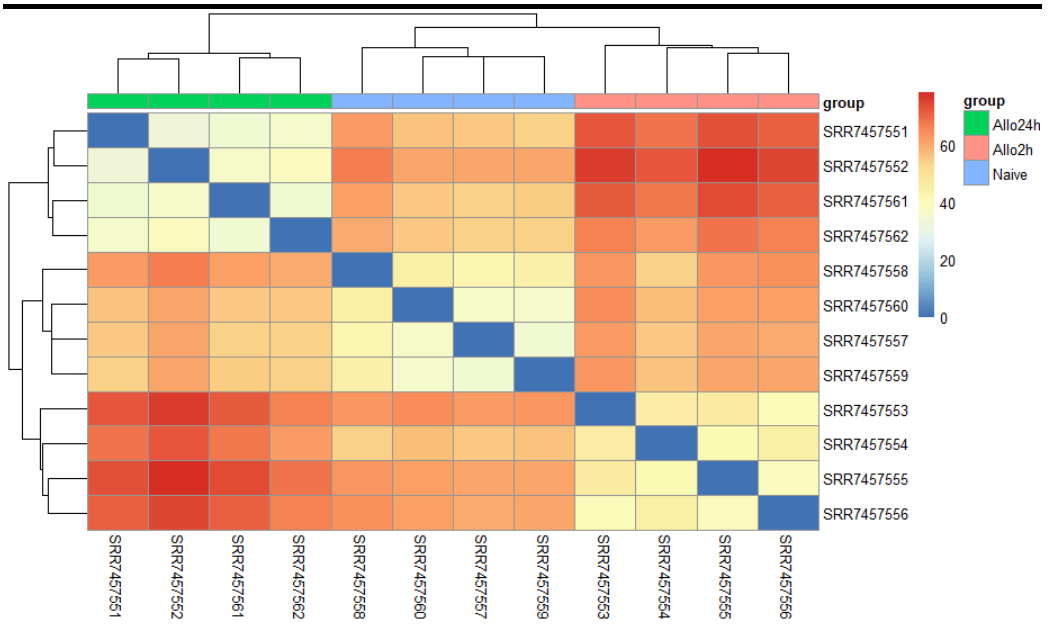


Figure 3: Heatmap of sample distances for GSE116583.

Detecting Differentially Expressed Genes

Now that we are satisfied with the quality of our RNA-Seq samples, it is possible to query our `DESeqDataSet` for results. `DESeq2` has a function for this, conveniently called `results()`. It is important to define what comparison we want to find results for - here we utilise the `contrast` argument for the `results()` function. This argument takes a vector of three elements: the column of the `colData` that is to be used for the comparison, the numerator of the comparison to be made, and the denominator of the comparison to be made, in that order. It is typical to use the control samples for the denominator - in this way, when the treated samples have higher expression, we see a positive fold change in our results, which is intuitive.

`DESeq2` results are reported in a table (a `DataFrame`, which is subtly different from both a `data.frame` and a `tibble` - like many modern programming languages, `R` is cursed with multiple overlapping implementations of the same basic idea). Each row in the results table represents a gene (identified by an Ensembl Gene ID in our experiment), and contains 6 observations.

Observation	Description
baseMean	The arithmetic mean of the normalised counts for this gene in *all* samples (not just those involved in this contrast)
log2FoldChange	The difference in expression between the mean normalised counts of the two conditions involved in the described contrast. This is calculated as the $\log_2$ of the ratio of normalised counts (numerator/denominator)
lfcSE	The standard error of the log2FoldChange
stat	The Wald test statistic
pvalue	The Wald test p-value, derived from the test statistic
padj	The p-value adjusted for multiple tests - this adjustment is carried out using the Benjamini-Hochberg correction

This results table is essentially "tidy" and can be used in the kind of plotting functions we explored in practical 05 - for example, the log2FoldChange and padj can be used to build a Volcano Plot.

Exercise 8.5

Estimated time: 15 minutes

- Run the following R code to carry out the steps described above:

```
> results_table = results(dds, contrast= c('Group',
> summary(results_table)
# filter any rows with an NA anywhere:
> results_tibble = as_tibble(results_table, rownames
> filtered_results = filter(results_tibble, complete
# Volcano plot (basic)
> filtered_results = mutate(filtered_results, logPVa
> ggplot(filtered_results, aes(x=log2FoldChange, y=l
        geom_point()
```

- Can you repeat the above for the second comparison inherent in this dataset (Allo2h vs Naive)?
- Is there any value in comparing Allo24h to Allo2h?

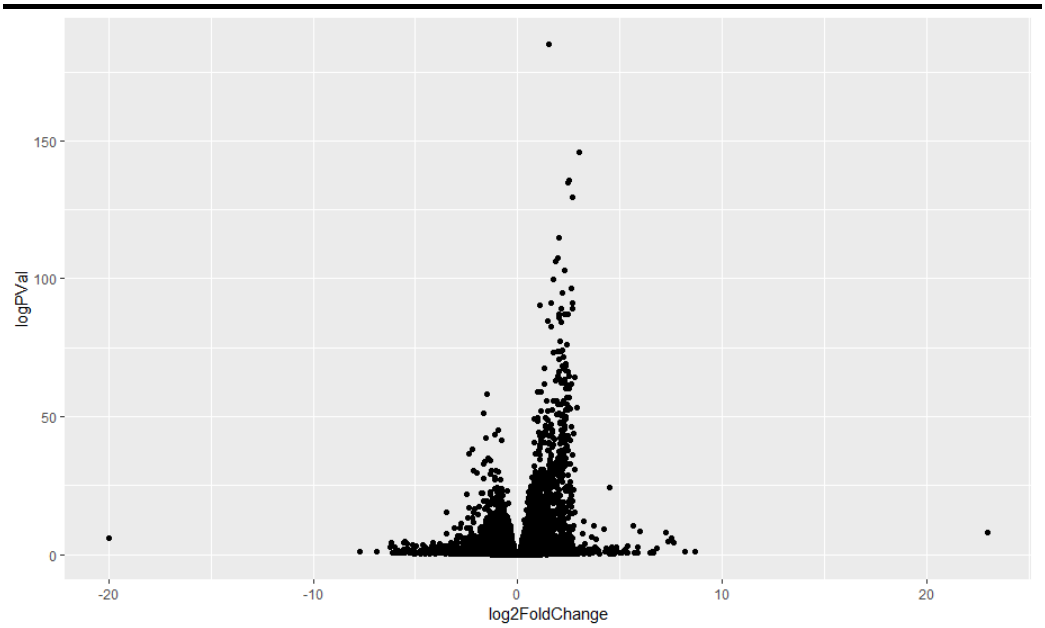


Figure 4: Basic 24h vs Naive Volcano Plot.

Annotating Results Table

We used the Gencode gene annotation when quantifying expression in this experiment. Therefore the rows of our count (and results) tables are identified by Ensembl Gene IDs. This is all very well, but human-readable gene names are more useful and informative in most situations.

There are many ways of retrieving richer annotation based on these Ensembl Gene IDs. All methods are complicated by the fact we must ensure that the version of Ensembl used for labelling thr genes in the first place (through Salmon), and the version used for producing the annotation in R must match - Ensembl obsoletes identfiers surprisingly frequently, and even a single version mismatch can cause IDs to go missing.

In producing the quantification with Salmon, we used Gencode release M31 which was published on 19th October 2022. We can see from this table: https://www.gencodegenes.org/mouse/releases.html that M31 is paired with Ensembl release 108 (20/10/2022). We an access this specific annotation in R using the `biomaRt` package, which we can use to retrieve lots of annotation information for our mouse genes.

`biomaRt` is a programmatic interface to Ensembl's BioMart database - a system for extracting tables of data from Ensembl.

In the final exercise of this practical, we will map the mouse Ensembl Gene IDs to a range of information from BioMart, and then filter the top 10 genes by adjusted p-value to discover their identity.

Exercise 8.6

Estimated time: 15 minutes

- Run the following R code to carry out the steps described above:

```
> ensembl108 = useEnsembl(biomart="ensembl", version
> ensembl108 = useDataset("mmusculus_gene_ensembl",
> annotation = getBM(attributes=c('ensembl_gene_id',
                                  'start_position', 'e
                                  'strand', 'gene_biot
                                  'description'),
                     filters = 'ensembl_gene_id', valu
                     mart = ensembl108)
> annot_results = left_join(filtered_results, annota
> annot_results = arrange(annot_results, padj)
> View(head(annot_results, 10))
> degs = filter(annot_results, abs(log2FoldChange) >
```

- Check that the annotation returned by BioMart matches the genes in your results table 1:1

- Can you think of a situation where this might not be the case?
- You can look at the attributes available through BioMart using the `listAttributes()` function - is there any other information you think might be useful?
- How many differentially expressed genes (DEGs) are there, based on the filter in the final line?

Summary

This practical has covered the essentials of working with RNA-Seq data in R, and particularly of using `DESeq2` to process such data and apply statistics to define differentially expressed genes. As mentioned at the outset, the exercises in this practical contain all of the code required to complete them to a basic standard. For the assessment, there is plenty of scope to improve on these versions.

In the final practical, we will explore ways of deriving biological meaning from the differential analysis presented here - we will look for functional coordination among the genes which change their expression, and look at methods for defining groups of genes which change together.